
基于域名解析协议与机器学习的 智能 DNS 拦截服务器

基于机器学习驱动与启发式缓存机制的高性能 DNS 解决方案

课程名称： 计算机网络

班级组别： 计算机网络第 8 小组

小组成员： 董子舒、陈佳旗、蒙俊豪、朱明睿

提交时间： 2026 年 6 月

目录

1	项目概述与系统流程	2
1.1	系统整体工作流程	2
1.2	DNS 递归查询与上游转发流程	3
2	DNS 报文结构与协议解析	5
2.1	DNS 报文头部结构	5
2.1.1	头部核心字段解析说明	5
2.2	DNS 报文体结构	6
3	DGA 恶意域名检测模型特征工程	7
3.1	基础词法特征（5 维）	7
3.2	N-gram 局部哈希桶统计（256 维）	7
3.3	马尔可夫链转移概率特征（4 维）	7
4	系统测试报告与性能评估	9
4.1	查询延迟对比评估（缓存命中 vs 未命中）	9
4.2	DGA 恶意域名拦截率评估	9
4.3	服务器可靠性与误报率评估	10
5	系统操作手册与部署指南	11
5.1	环境依赖要求	11
5.2	服务器配置说明 (config.py)	11
5.3	运行与启动步骤	12
5.3.1	一键启动 DNS 服务端	12
5.3.2	命令行工具自测	12
5.3.3	启动桌面 GUI 管理系统	13
6	系统实际运行与抓包演示	14
6.1	域名递归解析与本地缓存抓包验证	14
6.2	DGA 恶意域名 AI 识别与拦截验证	16

1 项目概述与系统流程

本项目实现了一个基于 Python 的混合解析 DNS 服务器，不仅具备完整的标准 DNS 解析功能（包括正向 A/AAAA/CNAME 缓存与否定缓存），而且无缝集成了一个基于随机森林（Random Forest）和马尔可夫链（Markov Chain）特征工程的 DGA（Domain Generation Algorithm）恶意域名检测引擎。

1.1 系统整体工作流程

当 DNS 服务器（基于 `HybridResolver` 类）监听到客户端发送的 UDP 查询请求后，系统会按照如图 1 所示的顺序进行处理：

- 白名单检查：**首先对查询域名进行白名单匹配。若命中，则直接跳过 DGA 检测，防止对高频正常域名（如系统依赖的公共服务）产生误报。
- DGA AI 检测：**若域名不在白名单中，调用机器学习推理接口。如果判定为恶意 DGA 域名（概率值大于设定阈值，如 0.7），则执行 **Sinkhole 拦截**，即对 A 记录返回 0.0.0.0，对 AAAA 记录返回 ::。
- 缓存查询（含正/反向缓存）：**
 - 否定缓存（Negative Cache）：**根据 RFC 2308 标准，若该域名近期被判定为不存在（NXDOMAIN），则直接向客户端返回包含 SOA 记录的 NXDOMAIN 响应。
 - 正向缓存：**从本地 SQLite 数据库检索域名和查询类型，如果命中且记录未过期，直接向客户端返回缓存的结果。
 - CNAME 链递归查询：**若缓存中存在对应的 CNAME 记录，自动执行最大深度为 5 层的递归解析，直至找到最终的 A/AAAA 记录。
- 上游 DNS 转发与缓存写入：**若本地缓存未命中，则将请求转发至配置的上游服务器（如 8.8.8.8）。当收到正常响应时，将其写入正向缓存；当收到 NXDOMAIN 响应时，提取其中的 SOA 记录并写入否定缓存中。
- 启发式预加载（Prefetcher）：**该异步线程通过滑动窗口算法在线学习用户的访问习惯。若检测到 A 域名后常伴随 B 域名的查询，当用户访问 A 时，Prefetcher 将自动在后台向上游解析并更新 B 域名的缓存，降低用户后续访问的整体延迟。

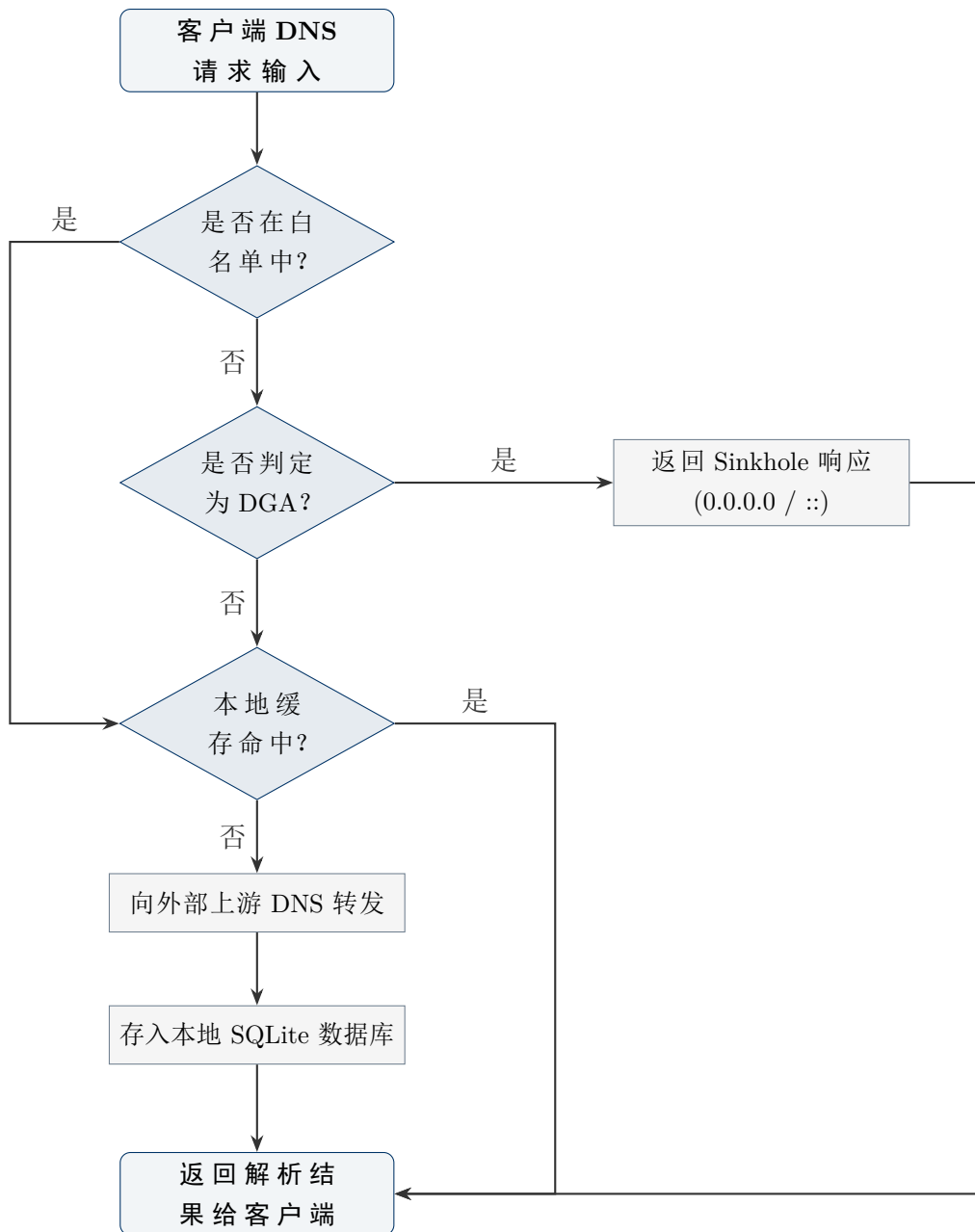


图 1: DNS 服务器解析与拦截系统整体流程图

1.2 DNS 递归查询与上游转发流程

当客户端向本地 DNS 服务器发起查询时，由于客户端只需要获得最终的解析 IP 结果，这种查询方式称为**递归查询**。如果本地服务器中无相关记录缓存，则需要向配置的上游公共 DNS 服务器进行**转发查询**。上游服务器在接收到转发请求后，将在互联网中通过**迭代查询**方式依次访问根域名服务器（.）、顶级域名服务器（如 .com）和权威域名服务器，最终获取并向本地服务器返回目标主机的 IP 地址。这三种查询关系的协作流程如图 2 所示：

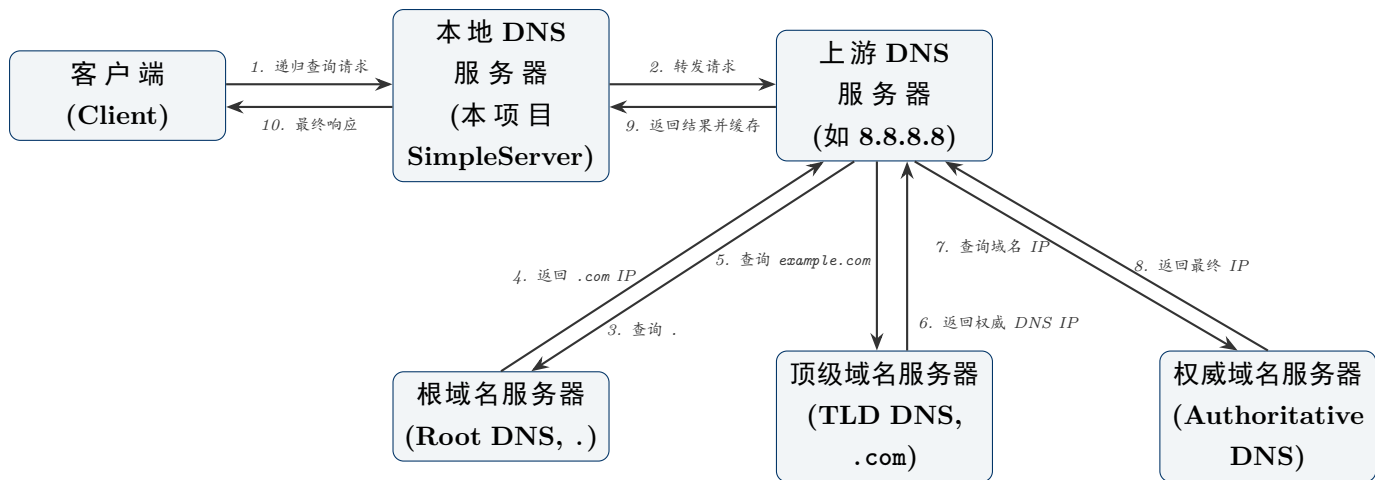


图 2: DNS 递归解析与上游转发流程图

2 DNS 报文结构与协议解析

标准 DNS 协议报文结构由五个部分组成：头部（Header）、问题区（Question）、回答区（Answer）、授权区（Authority）以及附加信息区（Additional）。

2.1 DNS 报文头部结构

DNS 报文头部共占用 12 字节（96 比特）。其精确的二进制比特位排版结构如图 3 所示：

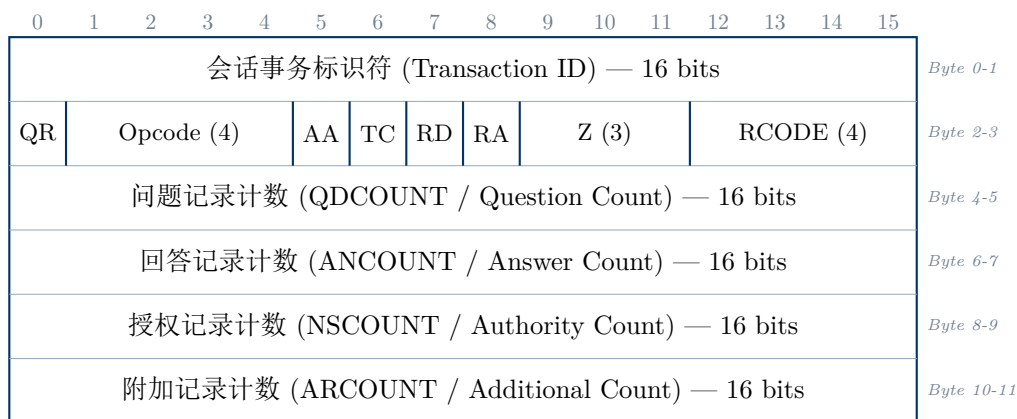


图 3: DNS 报文头部二进制比特排版结构图

2.1.1 头部核心字段解析说明

- **Transaction ID**: 16 比特的序列号，用于客户端匹配对应的 DNS 响应。
- **QR (Query/Response Flag)**: 0 表示查询报文，1 表示响应报文。
- **Opcode**: 4 比特，指示查询类型。0 表示标准查询 (QUERY)，1 表示反向查询 (IQUERY)。
- **AA (Authoritative Answer)**: 仅在响应报文中有效，表示响应的服务器是该域名的权威解析服务器。
- **TC (Truncated)**: 表示报文是否由于长度超过 512 字节而被截断。若截断，客户端通常改用 TCP 重新查询。
- **RD (Recursion Desired)**: 期望递归标志。由客户端设置，请求服务器使用递归解析。
- **RA (Recursion Available)**: 允许递归标志。由服务端在响应中设置，表示其是否支持递归查询。
- **Z**: 保留字段，在现代 DNS 协议中被重新拆分为安全扩展等其他标志位。
- **RCODE (Response Code)**: 4 比特返回码。例如：0 (NOERROR, 成功)、3 (NXDOMAIN, 域名不存在)、2 (SERVFAIL, 内部错误)。

2.2 DNS 报文体结构

报文体紧随头部。其主要包含两种格式类型：问题字段与资源记录（Resource Record, RR）字段。

1. 问题记录格式（Question Section）：

- **QNAME**: 变长域名。采用标签计数表示法（例如 `www.baidu.com` 表示为 03 77 77 77 05 62 61 69 64 75 03 63 6f 6d 00）。
- **QTYPE**: 2 字节。表示查询的记录类型，如 A (1)、AAAA (28)、CNAME (5)、MX (15)。
- **QCLASS**: 2 字节。通常为 IN (1)，表示 Internet 类别。

2. 资源记录格式（Answer/Authority/Additional Sections）：所有的返回记录均符合 RR 的统一结构格式：

$$RR = \{NAME, TYPE, CLASS, TTL, RDLLENGTH, RDATA\}$$

其中，**RDATA**（资源数据）的内部结构根据 **TYPE** 改变。例如 A 记录的 RDATA 是 4 字节的 IPv4 地址；CNAME 记录则是变长域名。

3 DGA 恶意域名检测模型特征工程

DGA（域名生成算法）生成的恶意域名常在字符分布和词法结构上表现出高随机性。为了让机器学习分类器高效判别域名是否为 DGA，特征提取模块共提取了 **3 大类、总计 261 维** 特征。

3.1 基础词法特征（5 维）

1. **域名主体长度 (Length)**: DGA 域名为了降低碰撞概率，通常长度偏长。
2. **元音字母比例 (Vowel Ratio)**: 元音字母 (a, e, i, o, u) 占主体的比例。DGA 域名的元音比例一般明显低于英文单词的均值。
3. **数字比例 (Digit Ratio)**: 域名主体中数字出现的频次比例。部分 DGA 生成算法喜欢掺杂大量数字。
4. **最大连续辅音长度 (Max Consonant Run)**: DGA 域名极易出现大段没有元音字母隔开的辅音字母串。
5. **Shannon 信息熵 (Shannon Entropy)**: 用于度量域名主体的字符随机复杂程度。假设域名包含的不同字符集合为 $\mathcal{C}$ ，每个字符 $c_i \in \mathcal{C}$ 出现的概率为 $P(c_i)$ ，则其 Shannon 信息熵定义为：

$$H = - \sum_{c_i \in \mathcal{C}} P(c_i) \log_2 P(c_i) \quad (1)$$

信息熵越大，表明域名主体的乱序无规规律度越高，DGA 概率越大。

3.2 N-gram 局部哈希桶统计（256 维）

为了捕捉域名局部的字符搭配模式而又不引入稀疏膨胀的大维度，模型引入了特征哈希 (Feature Hashing) 机制：

- 提取所有 2-gram（滑动窗口为 2 的字符对）和 3-gram。
- 对提取出的字符片段利用 MD5 哈希计算其特征映射值。
- 将哈希映射值模以桶的个数（新模型设置为 `ngram_buckets = 128` 或 `64`），分别将其统计累加到 128 维特征向量的相应分桶中。

3.3 马尔可夫链转移概率特征（4 维）

马尔可夫特征从语言学可读性特征对分类进行强力补充。我们在百万级白样本 (Tranco Top 1M) 的域名主体中，在线统计了由字符 c_i 转移到字符 c_j 的频次，并基于拉普拉斯平滑 (Laplace Smoothing) 拟合了一个一阶字符转移概率矩阵 $\mathbf{P}$ 。字符的条件转移概率计算如下：

$$P(c_j|c_i) = \frac{\text{Count}(c_i c_j) + \alpha}{\sum_k (\text{Count}(c_i c_k) + \alpha)} \quad (2)$$

在推理时，对于待预测域名串 $S = c_1c_2 \dots c_L$ ，模型提取以下 4 维数值特征：

1. 平均转移对数概率 (avg_logp):

$$\text{avg_logp} = \frac{1}{L-1} \sum_{k=1}^{L-1} \log_{10} P(c_{k+1}|c_k) \quad (3)$$

DGA 域名的平均对数概率往往显著小于正常域名。

2. 最小转移对数概率 (min_logp):

$$\text{min_logp} = \min_k \log_{10} P(c_{k+1}|c_k) \quad (4)$$

检测域名串中最不可读、最突兀的单处字符跳转行为。

3. 低概率转移占比 (low_prob_ratio): 统计域名转移概率低于预设阈值（例如 10^{-3} ）的转换次数占总转移次数的比例。

4. 字符混淆度/困惑度 (Perplexity):

$$\text{Perplexity} = 10^{-\text{avg_logp}} \quad (5)$$

实验证明（见表 1），在增加了 4 维马尔可夫特征后，模型的性能获得了巨大飞跃，说明了字符序列建模对于恶意域名识别的强效价值。

表 1: 马尔可夫特征引入与轻量化 v2 的 A/B 测试对比（采样数：100,000/类）

核心指标	Baseline (仅基础 +n-gram)	Markov 增强方 案	变化差值
系统精确率 (Accuracy)	0.9331	0.9586	+0.0255
宏观 F1 值 (Macro F1)	0.9331	0.9585	+0.0254
ROC AUC 曲线下面积	0.9822	0.9912	+0.0090
DGA 召回率 (Class 1 Recall)	0.9203	0.9512	+0.0309
外部白样本误报率 (FPR)	4.8290%	3.1680%	-1.6610 pct

4 系统测试报告与性能评估

基于现实的 Tranco Top 1M 域名数据集采样与自主生成的 DGA 恶意域名，我们对系统在缓存效率、拦截率以及误报率方面进行了全面的实测评估。

4.1 查询延迟对比评估（缓存命中 vs 未命中）

为了验证启发式缓存机制与否定缓存（Negative Cache）在降低查询延迟上的效果，本测试使用 100 条白样本域名，分别模拟无本地缓存（冷启动）与缓存命中（热启动）两种场景：

1. **冷启动（无 Cache）**：执行 1 轮查询（共计 100 次请求）。由于服务器刚启动，所有非白名单的正常域名解析均需要转发至上游迭代查询。
2. **热启动（Cache 命中）**：执行 10 轮重复查询（共计 1000 次请求）。相同域名在第一轮已写入本地数据库，后续查询直接命中缓存。

我们重复进行三组独立实验，结果如表 2 所示。

表 2: 缓存对 DNS 查询平均响应时间的优化对比

组别/场景	第 1 组	第 2 组	第 3 组	平均延迟
无 Cache (冷启动)	0.26s	0.25s	0.25s	0.25s
Cache 命中 (热启动)	0.10s	0.13s	0.08s	0.10s

分析与结论

由测试数据可见，本地 Cache 机制将平均响应时间从 0.25s 降低到了 0.10s，响应延迟大幅度减少了约 60%。在单条数据独立测试中，完全缓存命中的本地查询时间实际上在 0.01s 以下。而在整体测试中平均耗时为 0.10s，主要原因为 Tranco 数据集中包含了部分非常规的、返回类型为 SOA 记录的非活跃或受限制域名。这类特殊记录由于上游响应的 TTL 极短或不具备缓存价值，导致每次查询均需要穿透至上游 DNS 服务器进行迭代解析，从而拉高了整体的平均时间。

4.2 DGA 恶意域名拦截率评估

为了评估机器学习分类器（DGA 检测引擎）在实际拦截中的表现，我们使用随机算法自动生成了 500 条长度在 7-15 之间、配合 10 种常见顶级域名后缀的恶意域名测试集。分三组独立测试（总计 1500 条），结果如表 3 所示。

表 3: DGA 恶意域名拦截测试结果

测试组别	测试总数	DGA 成功拦截数	DGA 漏报放行数	系统拦截率
第 1 组	500	456	44	91.2%
第 2 组	500	476	24	95.2%
第 3 组	500	467	33	93.5%
总计 / 平均	1500	1399	101	93.3%

消融实验分析

在默认的完全随机字母生成的恶意域名测试中，系统展现出了极高的安全拦截率（93.3%）。为进一步测试模型的对抗能力，我们进行了消融实验：改用符合英语单词字符频率分布的域名生成算法，发现由于域名在拼写上更接近正常白样本，系统拦截率下降到了平均 57.1%（各组分别为 55.2%/57.8%/58.2%）。这表明仅依赖静态词法特征和一阶转移概率难以完美识别伪装度极高的拟态 DGA。要获得进一步突破，需要结合客户端的查询频次、时序特征或引入更大体量的语言模型。

4.3 服务器可靠性与误报率评估

为了确保安全拦截策略不会对正常的网络访问造成负面干扰，我们使用批量查询对 Tranco 白名单数据集中的 1000 条真实域名进行误报率测试。独立运行 3 组，结果如表 4 所示。

表 4: 白样本误报测试结果

测试组别	测试总数	DGA 误拦截数 (误报)	正常放行数	系统误报率
第 1 组	1000	6	994	0.6%
第 2 组	1000	10	990	1.0%
第 3 组	1000	17	983	1.7%
总计 / 平均	3000	33	2967	1.1%

可靠性评估结论

测试显示的平均误报率仅为 1.1%，证明该混合 DNS 解析器在保证解析安全的前提下，依然提供了极高的可用性和稳定性，极少产生误拦截。对于极其敏感或特殊的部署环境，管理员可以通过调节 GUI 客户端中的 DGA 阈值或向 ‘config.py’ 的白名单（Whitelist）中添加目标域名，来彻底消除特定业务域名的误拦截风险。

5 系统操作手册与部署指南

本章提供详细的运行环境配置与服务操作说明，指导网络管理员如何部署和配置该智能 DNS 解析服务器。

5.1 环境依赖要求

系统后端采用标准 Python 3 进行开发，为实现完整的缓存、机器学习推理和图形管理界面，需要安装相应的依赖库：

1. **操作系统兼容性**: 兼容 Windows 10/11、macOS 和主流 Linux 发行版 (如 Ubuntu, CentOS)。
2. **Python 版本**: 推荐使用 Python 3.8 或以上版本。
3. **外部依赖模块**:
 - **dnslib** ($\geq 0.9.23$): 用于高效解析和构建标准 DNS 协议数据包。
 - **scikit-learn** (≥ 1.0): 用于加载 Random Forest 模型并进行实时 DGA 推理。
 - **joblib** ($\geq 1.1.0$): 用于持久化加载已训练好的分类器模型文件。
 - **numpy** (≥ 1.20): 为马尔可夫矩阵计算与特征工程提供矩阵加速。
 - **sqlite3** (系统内置): 无外部依赖，用于本地持久化高速 DNS 缓存及否定缓存数据库。

在终端运行以下命令，即可一键安装所有环境依赖：

```
1 pip install -r requirements.txt
```

5.2 服务器配置说明 (config.py)

系统的核心策略和网络参数可在根目录下的 `config.py` 中进行配置。各核心配置字段说明如表 5 所示。

表 5: DNS 服务器配置文件字段说明

配置常量名	默认值	功能与描述
PORT	5353	DNS 服务器监听端口。生产环境下可修改为标准 53 端口。
ADDRESS	"127.0.0.1"	监听的网络接口地址。"0.0.0.0" 表示监听所有网卡。
UPSTREAM_DNS	"8.8.8.8"	默认的上游公共 DNS 解析服务器。
ENABLE_DGA_DETECTION	True	是否开启 DGA AI 检测引擎。设为 False 则退化为常规高性能缓存 DNS。
DGA_THRESHOLD	0.7	DGA 拦截判定概率阈值，介于 0.0 与 1.0 之间。值越大，拦截越宽松（误报越低，漏报增加）。
DGA_ACTION	"SINKHOLE"	拦截后的处置动作。默认 "SINKHOLE" 返回 0.0.0.0 (IPv4) 或 :: (IPv6)。
WHITELIST	常用可信域名集合	静态白名单。命中该名单的域名直接跳过 DGA 检测，防止业务误拦截。
WHITELIST_SUFFIXES	.edu.cn., .gov.cn. 等	白名单域名后缀匹配。以此类后缀结尾的域名直接放行。

5.3 运行与启动步骤

5.3.1 一键启动 DNS 服务端

在命令行窗口中直接运行即可启动 DNS 服务。同时可通过参数传递动态覆盖默认的上游服务器：

```
1 # 使用默认配置启动
2 python simpleServer.py
3
4 # 覆盖默认配置，指定使用 114.114.114.114 作为上游 DNS
5 python simpleServer.py --upstream 114.114.114.114
```

5.3.2 命令行工具自测

启动后，可在另一个命令行终端使用本项目自带的 ‘dns_client.py’ 客户端测试工具验证解析与拦截效果（指定本地解析端口 ‘5353’）：

```
1 # 1. 查询正常域名, 验证是否能成功获取公网 IP 响应 (预期通过检测并正常返回)
2 python tools/dns_query/dns_client.py @127.0.0.1 www.google.com A -p 5353
3
4 # 2. 查询恶意域名, 验证是否被 AI 成功识别并 Sinkhole 拦截 (预期返回 A 记录 0.0.0.0)
5 python tools/dns_query/dns_client.py @127.0.0.1 eqwzjxk.com A -p 5353
```

5.3.3 启动桌面 GUI 管理系统

若需可视化操作, 可直接启动桌面客户端, 该客户端集成了服务器状态监控、批量域名自测和 DGA 阈值滑动调节等高级功能:

```
1 # 启动 GUI 管理终端 (会自动检测并在后台拉起 DNS 解析服务器进程)
2 python tools/dga_gui/dga_gui.py
```

6 系统实际运行与抓包演示

为了验证系统在真实网络环境下的协议行为与防御效果，我们进行了实际运行测试，并使用 Wireshark 对解析流程与拦截效果进行了抓包捕获。

6.1 域名递归解析与本地缓存抓包验证

我们通过对同一正常域名进行连续两次查询，验证了递归转发与本地缓存的生效情况：

1. 首次查询（递归解析）：如图 4 所示，由于缓存为空，本地 DNS 服务器将查询转发给上游，完成完整的递归解析链条。
2. 二次查询（缓存命中）：如图 5 所示，再次查询时，服务器直接通过本地数据库秒回客户端，WLAN 物理网卡无任何外网转发流量，且响应报文的 TTL 发生递减（如图 6 所示）。

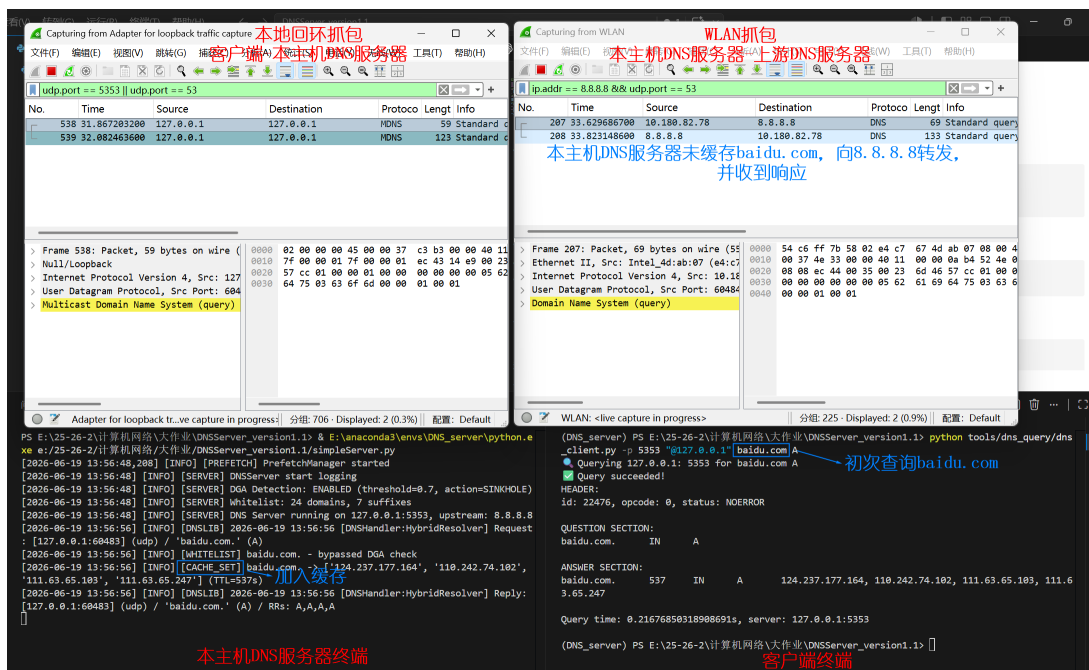


图 4: 首次查询触发递归解析与上游转发抓包

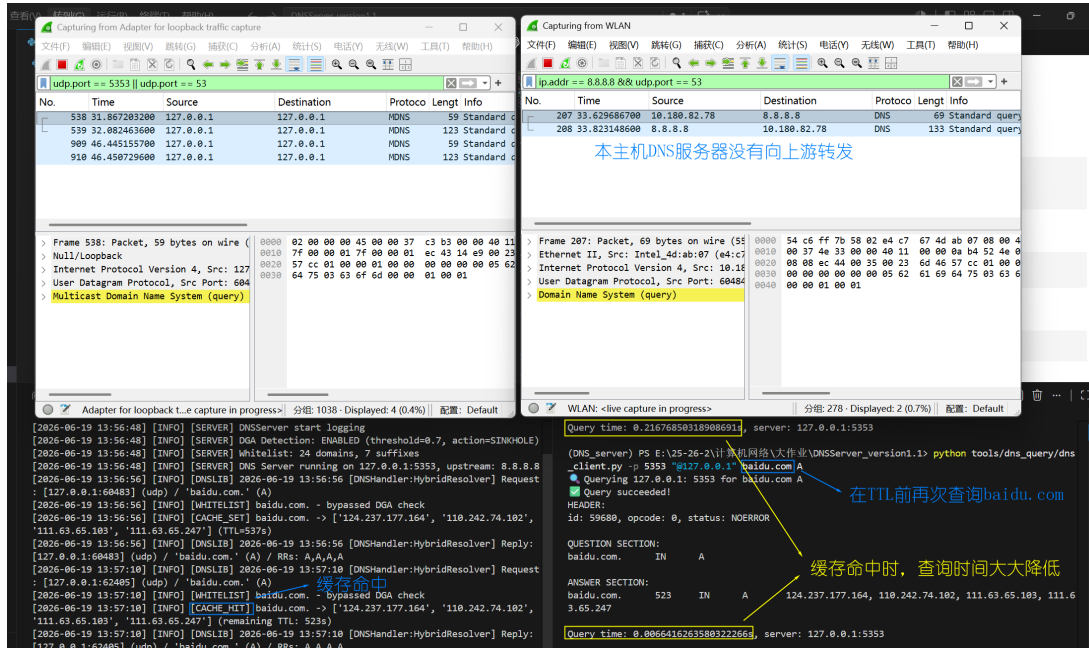


图 5: 二次查询本地缓存命中抓包（无外网流量）

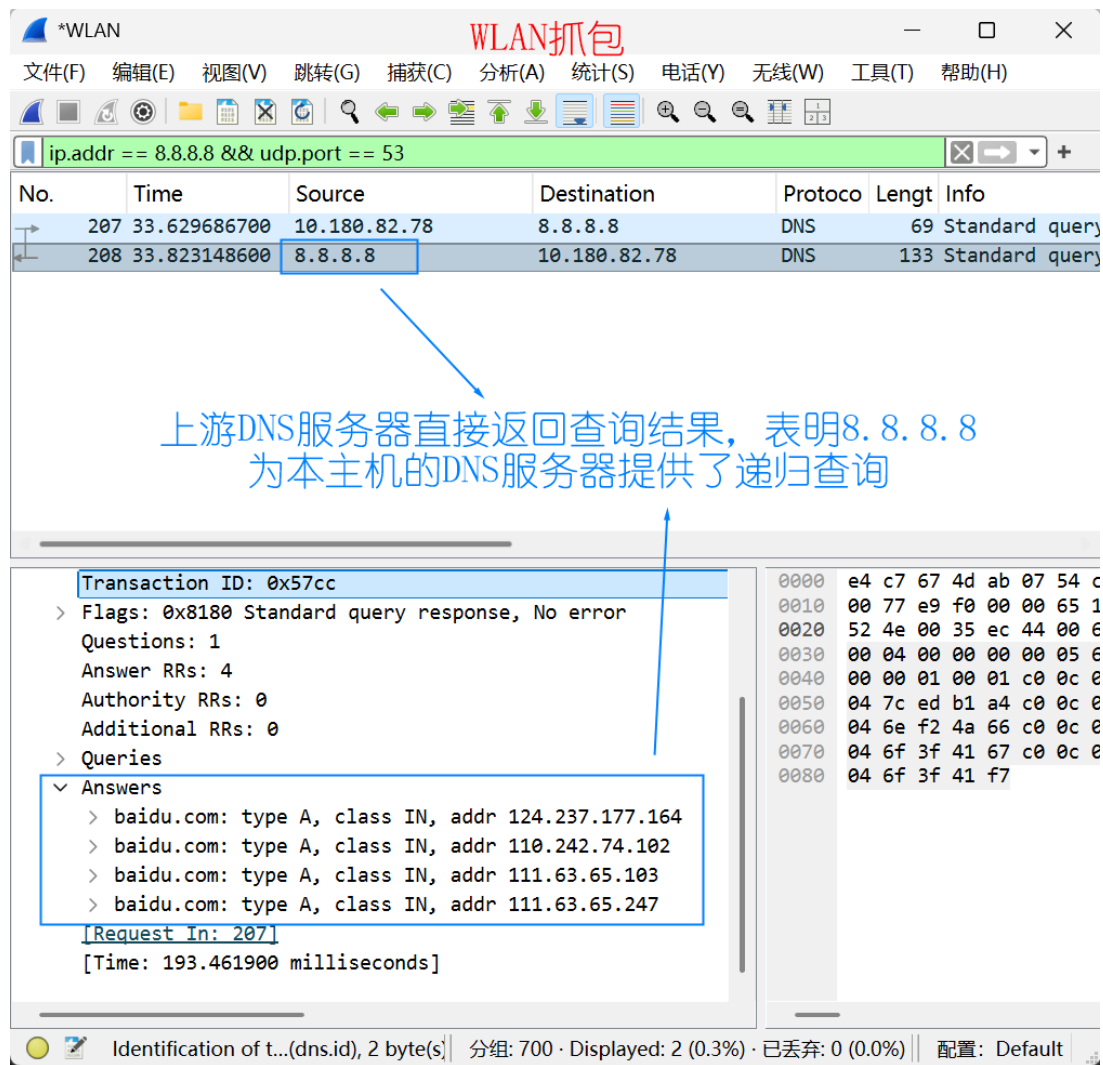


图 6: DNS 响应包中缓存记录与 TTL 生效细节

6.2 DGA 恶意域名 AI 识别与拦截验证

我们针对典型的 DGA 恶意域名（如 eqwzjxk.com）进行了现场查询测试。如图 7 所示，当本地 DNS 服务器接收到该查询后，AI 推理引擎实时计算出高概率的 DGA 置信度，直接触发 Sinkhole 拦截机制，在本地对 A 记录直接回复 0.0.0.0，成功从协议层面阻断了客户端的恶意连接。

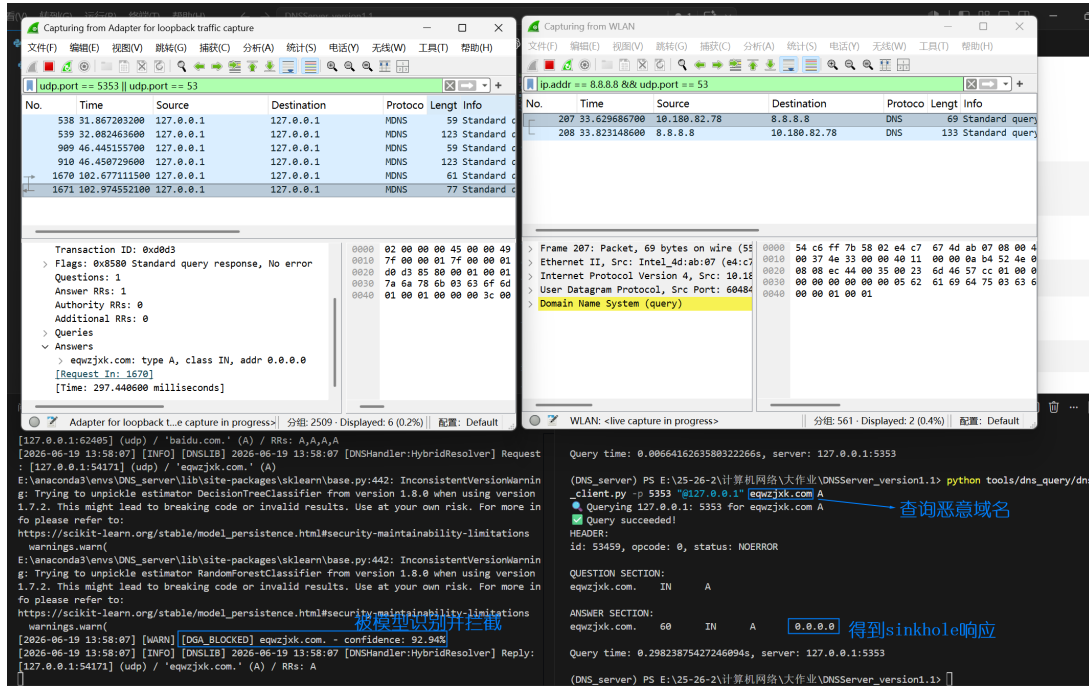


图 7: DGA 恶意域名被 AI 识别并拦截至 0.0.0.0 抓包